

LLMs for Data Science

大语言模型与数据科学工作流

清华大学统计与数据科学系 | 俞声 | 2026.4.29

第一章 一个值得铭记的日期

2022 年 11 月 30 日，OpenAI 正式发布了 **ChatGPT**。这是基于 GPT-3.5 系列、经过指令微调（Instruction Fine-tuning）和人类反馈强化学习（RLHF）训练的对话模型。发布后数天内用户量突破百万，将大语言模型从学术界推向了每一个数据科学家的日常工具箱。

这一天被很多人称为「AI 的 iPhone 时刻」——不是某项技术的渐进改进，而是让普通人第一次真切感受到 AI 能力的断层式跃迁。

1.1 LLM 如何冲击了数据科学工作

第一层冲击：淘汰。2023 年之前许多 NLP 研究成果被快速淘汰——几乎每个文本处理领域的研究者都经历过这种阵痛。

第二层冲击：涌现。同时，前所未有的新可能大量涌现。以下是一个真实案例：

真实案例：EHR 电子病历结构化

旧做法（极难实现，从未有人完整做到）：

格式清洗 → 命名实体识别（NER）→ 词义消歧 → 实体链接
→ 断言分析 → 解剖位置识别 → 数值与单位推断
任意一个组件出错都会影响整体，整套系统极易崩溃。

新做法：训练一个 LLM，端到端完成全流程。

训练数据也由 LLM 自动标注，支持跨语言转换。

→ 这两件事在 2023 年之前都不可能实现。

核心变化

LLM 把「语义理解」变成了一个可调用的函数——你不再需要手工编写理解规则，只需要描述你想要什么。

第二章 LLM 简史：从 LSTM 到 ChatGPT

要理解 LLM 为什么这么强大，需要了解它站在哪些巨人的肩膀上。

年份	模型	参数量	核心贡献
1997	LSTM	—	早期序列语言模型，2007 年起流行，能力有限

2013	Word2Vec	—	用向量表示词语语义，证明「无监督预训练」有用，但本身不是语言模型
2018	BERT	3.4 亿	预训练 Transformer 编码器，引入「预训练→微调」范式，全面超越前代
2020.6	GPT-3	1750 亿	引入「上下文学习（in-context learning）」与「提示词（prompt）」概念
2022.4	PaLM	5400 亿	引入「思维链（chain-of-thought）」，显著提升多步推理能力
2022.11	ChatGPT	约 20 亿	对齐人类价值观，指令跟随，多轮对话，实用化 AI 正式到来

2.1 几个关键概念

预训练（Pre-training）

在海量无标注文本上让模型学习「预测下一个词」。这一过程将人类书写的知识压缩进数十亿个参数中，使模型获得丰富的世界知识。

微调（Fine-tuning）

在预训练基础上，用少量特定任务的标注数据继续训练，让模型行为符合具体需求（如对话、翻译、分类）。

上下文学习（In-context Learning）

不更新模型参数，只在提示词里给几个示例，模型就能举一反三——这也是为什么**写好 prompt** 如此重要。GPT-3 的关键发现：模型大到一定程度，这个能力自然「涌现」出来。

思维链（Chain-of-Thought, CoT）

在提示词里加上「Let's think step by step」或给出分步推理示例，模型会模仿这种推理方式，**复杂问题准确率显著提升**。PaLM 的关键发现。

第三章 什么时候用 LLM，什么时候不用？

3.1 传统方法 vs LLM：核心差异

任务	传统方法	LLM 方法	谁更好？
修复拼写/错别字	字典查找 / 规则	语义纠错，理解上下文	LLM（理解意图）
类别归一化	人工维护映射表	语义推断等价表达	LLM（无需维护）
从文本提取结构	正则表达式	语义理解各种格式变体	LLM（格式多样时）
处理歧义	困难，规则爆炸	借助世界知识自然消歧	LLM
数值缺失值填充	统计方法（均值/插值）	不适合	传统方法

精确数学计算	确定性	会出错	传统方法
--------	-----	-----	------

判断标准

任务需要「世界知识」或「语义理解」 → 用 LLM

任务需要「精确逻辑」或「确定性结果」 → 用传统方法

3.2 好的使用场景

- **杂乱文本字段**：地址、职位名称、诊断描述、用户评论等
- **类别不一致**：「高血压」「血压升高」「原发性高血压」需要统一
- **多语言归一化**：中英文混杂的医学术语
- **半结构化文本 → 结构化表格**：从自由文本病历中抽取字段

3.3 不好的使用场景

- **数值操作**：缺失值填充、归一化、缩放——用 pandas/NumPy
- **严格可复现性**：LLM 本质随机，同样输入不保证同样输出
- **精确逻辑**：需要确定性结果的规则判断
- **超大规模廉价流水线**：千万条×每条有成本，不可接受

3.4 LLM 在数据流水线中的位置

LLM 是数据流水线的清洗层，夹在原始数据和后续分析之间：



第四章 核心编程模式

4.1 心智模型：LLM 是一个数据变换函数

在代码层面，LLM 就是一个函数：给它一段文字，它返回一段文字。心智模型是：

`LLM(prompt(text))`

`prompt(text)` 是把数据格式化提示词的模板函数；`LLM()` 是调用模型的接口函数。两者合在一起，LLM 就成了一个「语义数据变换函数」，用来替代传统的手工规则函数。

4.2 规范提示词的五部分组成

一个设计良好的提示词包含以下五部分，缺一不可：

部分	作用	作业中的对应写法
----	----	----------

system prompt	设定 LLM 的专家角色	你是中文医学信息学专家，熟悉 UMLS 语义类型体系
instruction（指令）	明确任务目标	任务：对每个医学概念 (1) 判断语义类型，(2) 生成标准名称
constraints（约束）	限定合法输出范围	语义类型只能从以下列表中选择，不得创造新类型：...
input slot（输入槽）	插入本次要处理的数据	ID {item['id']}: {terms_str}
output format（输出格式）	规定返回的结构	严格 JSON 数组，不含任何解释或代码块标记

4.3 作业中的提示词实现

```
SYSTEM_PROMPT = f"""你是中文医学信息学专家，熟悉 UMLS 语义类型体系。 # ① system prompt: 角色
任务：对每个医学概念 (1) 判断语义类型，(2) 生成标准名称。 # ② instruction: 任务
语义类型只能从以下列表中选择，不得创造新类型： # ③ constraints: 限定合法值
{TYPES_LIST_STR}
```

判断语义类型注意： # 更多约束：帮模型处理模糊情况

- 中医治法（如健脾疏风）归 Therapeutic or Preventive Procedure
- 中医证型（如气阴两虚）归 Disease, Syndrome or Pathologic Function
- 拿不准的优先归类，只有纯数字、人名才选「其它」

输出格式：严格 JSON 数组，不含任何解释或代码块标记： # ④ output format

```
[{"id": <整数>, "semantic_type": "<类型>", "standard_name": "<名称或 null>"}, ...]
"""
```

```
def make_prompt(batch):
    lines = [f'请处理以下 {len(batch)} 个医学概念：\n'] # 告诉模型本次有几个概念
    for item in batch:
        top_terms = sorted(item['terms'], key=lambda x: x[1], reverse=True)[:8]
        terms_str = ','.join(t[0] for t in top_terms) # 拼成「术语A、术语B、...」
        lines.append(f'ID {item["id"]}: {terms_str}') # ⑤ input slot: 每行格式 ID 0: 高血压、血压升高...
    return '\n'.join(lines)
```

为什么只取 TOP-8?

每个医学概念可能有几十个等价术语（同义词）。全部塞进去会让 prompt 太长。
token 越多 → 花费越多 → 响应越慢。按权重取 TOP-8，保留最重要的信息同时控制成本。

4.4 控制输出格式：三个层级

- **Level 1 – 纯文本：**「请把这个职位名称规范化」→ 模型可能回答一段话，无法可靠解析
- **Level 2 – 要求 JSON：**Return JSON: {"title": "<normalized>"} → 通常能直接 json.loads()
- **Level 3 – 严格 schema（最佳实践）：**固定键名、禁止附加解释、给出模板

常见问题：Markdown 代码块污染

当 messages 里有 system role 时，模型认为自己在跟人聊天，会把 JSON 包裹成：

```
```json
[{"id": 0, "semantic_type": "Disease...", ...}]
```
```

这会让 json.loads() 直接报错。解决方法见下方代码。

```
raw_text = raw_text.strip() # 去掉首尾空白和换行
if raw_text.startswith('```'): # 如果以 ``` 开头（可能是 ```json）
    raw_text = raw_text.split('\n', 1)[1] # 删掉第一行
if raw_text.endswith('```'): # 如果以 ``` 结尾
    raw_text = raw_text.rsplit('```', 1)[0] # 删掉最后的 ```
return json.loads(raw_text) # 现在可以安全解析了
```

第五章 LLM 的随机性与 Temperature

5.1 为什么输出是随机的？

LLM 不是查字典——它在每一步都从一个概率分布中**采样**下一个 token（词片段）。这个分布由温度参数 T 控制：

$$P(x_i) = \exp(z_i / T) / \sum_j \exp(z_j / T)$$

T → 0：每次都选概率最高的 token，输出趋向固定。 T 越高：概率分布越均匀，输出越多样（也越容易出错）。

| Temperature | 效果 | 适用场景 |
|---------------|--------------|-------------------|
| T = 0 | 几乎固定，每次相同 | 极高一致性要求（但仍不能完全保证） |
| T = 0.1（作业设置） | 极低随机性，稳定且不死板 | 数据清洗、格式转换等数据科学任务 |
| T = 0.7 ~ 1.0 | 明显随机，输出多样 | 创意写作、头脑风暴 |

实践建议

作业里 `temperature=0.1` 是正确选择。

即使设为 0，也不能保证跨平台、跨版本完全相同的输出——不要依赖精确复现性。

始终对输出做格式校验（检查 `semantic_type` 是否在合法列表里），而不是假设模型一定正确。

第六章 第一个 LLM 程序：调用 API

6.1 通过服务商调用 LLM

大多数情况下我们不自己运行模型，而是调用服务商的 API（「Token 工厂」）。作业使用的是 **SiliconFlow**（国内平台，DeepSeek/Qwen 等模型），其 API 格式与 OpenAI 完全兼容——学会一种，换服务商只需改 URL 和 Model 名。

6.2 API Key：访问凭证，务必保管

```
# 正确做法：从文件读取，不硬编码
with open('hw9/my_api_key.txt', mode='r') as f:
    API_KEY = f.readline().strip() # strip() 去掉末尾换行符，否则认证会失败

# 错误做法：直接写在代码里
API_KEY = 'sk-xxxxxxxxxxxxxxxx' # 提交到 GitHub 会被自动扫描盗用！
```

API Key 泄露的后果

GitHub 上有专门的机器人扫描新提交的代码，发现 Key 后立刻盗用。

轻则配额被消耗殆尽，重则产生大额账单（有人因此收到数千美元的账单）。

正确做法：Key 写在独立文件，在 `.gitignore` 里排除，代码里只读文件。

6.3 配置：URL、Model、Headers

```
URL = 'https://api.siliconflow.cn/v1/chat/completions' #
/v1/chat/completions 是 OpenAI 格式固定路径
MODEL = 'deepseek-ai/DeepSeek-V3.2'

HEADERS = {
    "Authorization": f"Bearer {API_KEY}", # 格式固定：Bearer 空格 Key
    "Content-Type": "application/json" # 告诉服务器我发的是 JSON
}
```

6.4 最简版调用函数（同步）

先看同步（非异步）版本，理解请求结构：

```
import requests, json

def call_llm(prompt):
    data = {
        "model": MODEL,
        "messages": [
            {"role": "system", "content": SYSTEM_PROMPT}, # system: 全局角色设定
            {"role": "user", "content": prompt} # user: 本次要处理的数据
        ],
        "temperature": 0.1
    }
    response = requests.post(URL, headers=HEADERS, json=data)
    return response.json()["choices"][0]["message"]["content"] # OpenAI 格式固定提取路径
```

messages 的三种 role

"system": 系统提示，设定角色和行为规范，整个对话期间都生效

"user": 用户的提问或要处理的数据

"assistant": LLM 之前的回复（多轮对话时用，单次清洗任务一般不需要）

→ 作业同时用了 system 和 user，这也是为什么会出现 Markdown 代码块问题（system 让模型以为在聊天）

第七章 大规模调用：异步编程

7.1 LLM 调用很慢——串行处理行不通

每次 API 调用需要：建立网络连接 + 服务端排队 + LLM 推理，单次约 2~3 秒。

| 处理方式 | 1000 条÷20 条/批=50 批 | 时间估算 |
|------------|--------------------|--|
| 串行（一个接一个） | 第 1 批完成后才开始第 2 批 | $50 \times 3 \text{ 秒} \approx 2.5 \text{ 分钟}$ |
| 并发=5（作业设置） | 同时运行 5 批，某批完成立刻补新批 | $\approx 30 \text{ 秒}$ |
| 并发=50 | 同时运行全部 50 批 | $\approx 3 \text{ 秒}$ （但可能触发限流） |

7.2 为什么异步能解决这个问题

调用 LLM API 是典型的 **I/O 密集型任务**——大部分时间花在等待网络响应，CPU 其实是空闲的。异步编程的思路：**等待期间不浪费，去做别的事**。

| 任务类型 | 等待的是什么 | 加速方法 |
|-----------------|----------------|-----------------------|
| I/O 密集型（LLM 调用） | 网络响应（CPU 空闲等待） | asyncio 并发：等待时去处理其他请求 |

| | | |
|---------------|--------------|--------------------------|
| CPU 密集型（数值计算） | 计算本身（CPU 在算） | multiprocessing 并行：多核同时算 |
|---------------|--------------|--------------------------|

7.3 asyncio 核心概念

| 关键字/函数 | 一句话解释 | 作业中的用法 |
|------------------------|----------------------|---|
| async def | 定义协程函数（可以被暂停的函数） | async def call_llm_async(...) |
| await | 暂停当前协程，等待结果，期间运行其他协程 | await response.json() |
| asyncio.gather(*tasks) | 同时启动多个协程，全部完成后返回结果列表 | await asyncio.gather(*tasks) |
| asyncio.Semaphore(n) | 限制同时运行的协程数量不超过 n | asyncio.Semaphore(CONCURRENCY) |
| asyncio.run(main()) | 启动事件循环，整个程序只调用一次 | if __name__ == '__main__':
asyncio.run(main()) |

7.4 真并发 vs 假并发（课件例子）

```
import asyncio

async def task(name, seconds):
    print(f'{name} 开始')
    await asyncio.sleep(seconds) # 模拟等待 API 响应（异步，等待期间可运行其他协程）
    print(f'{name} 结束')

# 假并发: 每个 await 都等上一个完成才开始，总计 2+4+3+1 = 10 秒
async def main_wrong():
    await task('A', 2)
    await task('B', 4)
    await task('C', 3)
    await task('D', 1)

# 真并发: gather 同时启动全部，总计 max(2,4,3,1) = 4 秒
async def main_correct():
    await asyncio.gather(
        task('A', 2),
        task('B', 4),
        task('C', 3),
        task('D', 1)
    )

asyncio.run(main_correct()) # 程序入口，整个程序只调用一次
```


Jupyter 注意

`asyncio.run()` 在 Jupyter Notebook 里会报错: `RuntimeError: This event loop is already running`

原因: Jupyter 本身就运行在事件循环里, 不能再套一层。

解决: 在 Jupyter 里直接写 `await main()`, 不要包裹 `asyncio.run()`

7.5 Semaphore 限流

`asyncio.Semaphore(n)` 是一个计数器, 保证同时进入某段代码的协程数不超过 `n`。原理类似停车场: 只有 `n` 个车位, 满了就在门口等。

```
semaphore = asyncio.Semaphore(5) # 最多同时 5 个请求「在飞」

async with semaphore: # 进停车场（占一个车位），满了就在这里等
    ... # 执行 HTTP 请求
    # 离开停车场（释放车位，下一个等待的协程进来）
```

为什么不直接发 50 个并发请求?

API 服务商限制每分钟请求数 (Rate Limit), 超过会收到 `429 Too Many Requests` 错误。

`CONCURRENCY=5` 是速度 (不要太小) 和稳定性 (不要太大) 的平衡点。

7.6 完整异步调用函数

```
import aiohttp # aiohttp 是 requests 的异步版本

async def call_llm_async(session, prompt, semaphore):
    data = {
        "model": MODEL,
        "messages": [{ "role": "system", "content": SYSTEM_PROMPT },
                     { "role": "user", "content": prompt }],
        "temperature": 0.1
    }
    async with semaphore: # 限流: 同时最多 CONCURRENCY=5 个协程进入
        async with session.post(URL, json=data, headers=HEADERS) as response:
            result = await response.json() # 异步等待响应（等待期间让出控制权）
            raw_text = result["choices"][0]["message"]["content"] # 固定提取路径
            # 去掉 Markdown 代码块标记
            raw_text = raw_text.strip()
            if raw_text.startswith('```'):
                raw_text = raw_text.split('\n', 1)[1]
            if raw_text.endswith('```'):
                raw_text = raw_text.rsplit('```', 1)[0]
            return json.loads(raw_text) # 解析为 Python 列表并返回
```

7.7 主流程 main() 完整拆解

```
async def main():
    # 1. 加载数据
    samples = load_sample(INPUT_FILE, SAMPLE_SIZE, RANDOM_SEED)

    # 2. 切片分批: 每批 BATCH_SIZE=20 条, 共 50 批
    batches = [samples[i : i+BATCH_SIZE]
                for i in range(0, len(samples), BATCH_SIZE)] #
    range(0,1000,20)=[0,20,...,980]
    print(f'共 {len(samples)} 条, {len(batches)} 个批次')

    # 3. 并发调用
    semaphore = asyncio.Semaphore(CONCURRENCY)
    async with aiohttp.ClientSession() as session: # 共享 Session, 减少重复建连接的开销
        tasks = [call_llm_async(session, make_prompt(b), semaphore)
                  for b in batches] # 为每批创建协程任务 (此时还没开始运行)
        results = await asyncio.gather(*tasks) # *tasks 把列表展开为独立参数, 并发执行全部

    # 4. 写入结果
    with open(OUTPUT_FILE, 'w', encoding='utf-8') as fout:
        for batch, batch_result in zip(batches, results): # batch 和 result 一一对应
            id_to_res = {r['id']: r for r in batch_result} # 转字典, 按 id 查找 (防止顺序错乱)
            for item in batch:
                res = id_to_res.get(item['id'], {}) # get 而非 [], LLM 漏返 id 时不会
                record = {
                    'id': item['id'],
                    'line_no': item['line_no'],
                    'semantic_type': res.get('semantic_type', '其它'), # 没有就给默认值
                    'standard_name': res.get('standard_name')
                }
                fout.write(json.dumps(record, ensure_ascii=False) + '\n') # JSONL 格式, 中文不转义
```

第八章 成本意识

LLM API 按 token 计费 (大约 1000 token $\approx$ 750 个英文单词 $\approx$ 500 个中文字)。作业处理 1000 条, **事先估算成本**非常重要。

- **控制 prompt 长度**: 作业取 TOP-8 术语而非全部, 直接减少 input token 数量
- **批处理**: 每次 20 条打包成一个请求, 减少请求次数, 降低固定开销
- **缓存结果**: 把已处理的 (输入, 输出) 存到文件, 下次跳过相同输入
- **输入去重**: 如果 1000 条里有重复, 去重后只需调用更少次
- **选合适的模型**: DeepSeek-V3 比 GPT-4o 便宜约 10~20 倍, 对于清洗任务完全够用

第九章 生产级代码

作业代码可以运行，但还缺几个让它「在真实环境中不崩溃」的要素：

| 缺少什么 | 不加的后果 | 解决方案 |
|-------------|---------------------|-------------------------------|
| 异常处理 | 网络超时或 API 报错时整个程序崩溃 | try/except 包裹，捕获网络异常 |
| 重试机制 | 某批次一次失败就永久丢失 | 失败后退避等待再重试，最多 3 次 |
| 断点续传 | 程序中途崩溃后从头开始 | 启动时读已有结果，跳过已处理的 id |
| 日志（logging） | 只有 print，出错时难以追溯 | 用 logging 替代 print，写带时间戳的日志文件 |
| 输出校验 | LLM 返回的类型不在合法列表里 | 解析后检查每个字段，不合法置为「其它」 |

9.1 异常处理与重试

```
async def call_llm_async(session, prompt, semaphore, max_retries=3):
    for attempt in range(max_retries): # 最多重试 3 次
        try:
            async with semaphore:
                async with session.post(URL, json=data, headers=HEADERS) as
response:
                    if response.status == 429: # 触发限流：等待后重试
                        await asyncio.sleep(2 ** attempt) # 指数退避：1s →
2s → 4s
                        continue
                    if response.status != 200:
                        raise Exception(f'HTTP {response.status}')
                    result = await response.json()
                    # ... 解析并返回
        except asyncio.TimeoutError:
            await asyncio.sleep(2 ** attempt) # 超时也退避重试
    return None # 全部重试失败，返回 None 而不是崩溃整个程序
```

9.2 断点续传

```
# 在 main() 开头加入：
processed_ids = set()
if os.path.exists(OUTPUT_FILE): # 如果输出文件已存在，说明之前跑过一部分
    with open(OUTPUT_FILE, 'r') as f:
        for line in f:
            processed_ids.add(json.loads(line)['id']) # 记录所有已完成的 id
print(f'已处理 {len(processed_ids)} 条，跳过')
```

```
samples = [s for s in samples if s['id'] not in processed_ids] # 只处理还没完成的
```

9.3 日志系统 (logging)

用 logging 替代 print，好处：① 自动加时间戳；② 区分日志级别；③ 写入文件，程序关闭后仍可查看。

```
import logging
from datetime import datetime

def setup_logging():
    log_file = f"run_{datetime.now().strftime('%Y%m%d_%H%M%S')}.log" # 文件名带时间戳
    logging.basicConfig(
        level=logging.INFO, # 只记录 INFO 及以上 (DEBUG 太啰嗦，默认不记)
        format='%(asctime)s - %(levelname)s - %(message)s', # 格式：时间 - 级别 - 消息
        handlers=[
            logging.FileHandler(log_file, encoding='utf-8'), # 写入文件，程序崩溃后仍可查看
            logging.StreamHandler() # 同时打印到控制台
        ]
    )
    return logging.getLogger(__name__)

logger = setup_logging()

# 替代 print 的写法：
logger.info(f'共 {len(samples)} 条, {len(batches)} 个批次') # 替代 print
logger.warning(f'Rate limit, 等待重试 attempt={attempt}') # 有问题但不停止
logger.error(f'批次失败: {e}') # 某次操作失败了
```

9.4 日志级别速查

| 级别 | 使用时机 | 示例 |
|----------|------------|---|
| DEBUG | 调试细节，默认不显示 | logger.debug(f'处理 ID {id}') |
| INFO | 正常流程里程碑 | logger.info(f'完成 {i}/{total} 批') |
| WARNING | 有点不对但不影响运行 | logger.warning(f'Rate limit, 重试 {attempt}') |
| ERROR | 某次操作失败了 | logger.error(f'批次 {i} 失败: {e}') |
| CRITICAL | 程序可能无法继续 | logger.critical('API Key 无效，停止') |

第十章 快速参考

10.1 常见报错速查

| 报错 | 原因 | 解决 |
|--|-----------------------------|---------------------------------------|
| TypeError: string indices must be integers | result 是字符串不是字典，API 返回了错误响应 | 打印 response.json() 查看，通常是 Key 失效或余额不足 |
| JSONDecodeError | LLM 返回了带 ``json 的内容未去除 | 加上 strip Markdown 的三步后处理 |
| KeyError: 'choices' | API 返回了错误结构 | 打印完整 response.json()，查看 error 字段 |
| 429 Too Many Requests | 超出 API 速率限制 | 降低 CONCURRENCY，加重试+指数退避 |
| RuntimeError: event loop already running | 在 Jupyter 里用了 asyncio.run() | 改为直接 await main() |

10.2 作业调用链总图

| | |
|---|--|
| <pre>asyncio.run(main()) ├─ load_sample() ├─ samples[i:i+20 for i in range(0,1000,20)] ├─ asyncio.Semaphore(5) ├─ aiohttp.ClientSession() ├─ [call_llm_async(...) for batch in batches] └─ asyncio.gather(*tasks) ├─ make_prompt(batch) ├─ async with semaphore ├─ session.post(URL, ...) ├─ await response.json() └─ strip Markdown + json.loads() └─ zip(batches, results) → 写 JSONL 文件</pre> | <pre># 程序从这里开始 # 读文件，随机采 1000 条 # 切成 50 批 # 创建限流器（最多 5 个并发） # 创建共享 HTTP 连接池 # 创建 50 个协程任务 # 并发执行（最多 5 个同时跑） # 构造用户提示词 # 限流 # 发 HTTP 请求 # 异步等待响应 # 清理格式，解析 JSON # 对齐结果，逐行写入输出</pre> |
|---|--|

「如果你已读到这里，你已经是一名相当有能力的数据科学家了。」
—— 课件最后一页